But let's start from the beginning: the very construction of a Bloom filter. Only afterwards we'll examine in details `insert` and `contains`.

4.9.2 Constructor

Construction of a Bloom filter is pretty easy: we just have to initialize an array of bits, with all its elements set to `0`, and generate the set of `k` hash functions. We have seen that the implementation also involves some computation, but it's safe to mark all that part as constant time.

Creating and initializing the array requires, obviously, `O(m)` time, while generating each of the hash functions typically requires constant time, hence `O(k)` time overall is needed to generate the whole set.

The whole construction can be ultimately finished in `O(m+k)` steps.

4.9.3 Storing an Element

For each key to store, we need to produce `k` hash values, and flip a bit in each of the elements of the array indexed by those results.

We'll make the following assumptions:

- Storing a single bit requires constant time (possibly including the time needed for bitwise operations, in case we use compressed buffers to save space).
- Hashing a key `X` requires `T(X)` time.